

Pareto Conditioned Networks

Mathieu Reymond
Vrije Universiteit Brussel
Brussels, Belgium
mreymond@ai.vub.ac.be

Eugenio Bargiacchi
Vrije Universiteit Brussel
Brussels, Belgium
ebargiac@ai.vub.ac.be

Ann Nowé
Vrije Universiteit Brussel
Brussels, Belgium
ann.nowe@ai.vub.ac.be

ABSTRACT

In multi-objective optimization, learning all the policies that reach Pareto-efficient solutions is an expensive process. The set of optimal policies can grow exponentially with the number of objectives, and recovering all solutions requires an exhaustive exploration of the entire state space. We propose Pareto Conditioned Networks (PCN), a method that uses a single neural network to encompass all non-dominated policies. PCN associates every past transition with its episode's return. It trains the network such that, when conditioned on this same return, it should reenact said transition. In doing so we transform the optimization problem into a classification problem. We recover a concrete policy by conditioning the network on the desired Pareto-efficient solution. Our method is stable as it learns in a supervised fashion, thus avoiding moving target issues. Moreover, by using a single network, PCN scales efficiently with the number of objectives. Finally, it makes minimal assumptions on the shape of the Pareto front, which makes it suitable to a wider range of problems than previous state-of-the-art multi-objective reinforcement learning algorithms.

KEYWORDS

Multi Objective Reinforcement Learning; Pareto Front; Multi Policy

ACM Reference Format:

Mathieu Reymond, Eugenio Bargiacchi, and Ann Nowé. 2022. Pareto Conditioned Networks. In *Proc. of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2022)*, Online, May 9–13, 2022, IFAA-MAS, 9 pages.

1 INTRODUCTION

Decision makers acting in real-world problems often have to take into account multiple objectives. When maximizing one objective comes at the cost of another, the objectives are in conflict, and the decision maker must find a compromise between them. For example, maximizing the electricity output of a hydroelectric power plant comes at the expense of increased flooding risks downstream as well as irrigation deficits [4]. In the medical field, radiotherapy should generally maximize the destruction of cancer cells while minimizing the damage to the healthy surrounding tissue [10]. The optimal trade-offs might differ on a case-by-case basis, and they can be difficult to identify without a picture of all possible options.

Unfortunately, research work on optimizing a sequential decision problem often focuses on maximizing a single, scalar objective. For the aforementioned real-world problems, this means somehow combining the different objectives into a single target metric. A possible option is for the decision maker to ask an expert to manually

engineer a metric so to induce a specific behavior after optimization. However, this approach has several drawbacks. First, such work can be expensive and error-prone, as it often requires domain expertise and extensive tuning before the optimization process finally produces a behavior that satisfies the decision maker. By focusing on a single solution at a time, this process also fails to inform the decision maker of all the trade-offs that could be possible. Finally, the decision maker has no way to convey directly their actual preferences, instead having to rely on the expertise of the designer to construct a suitable metric. These issues can severely restrict the influence that the decision maker has on the process, thus possibly missing their preferred solutions.

In contrast we can directly learn the best compromises by using an explicitly multi-objective approach. Assuming that improving an objective is always preferable, we can build a set of all optimal trade-offs called a *Pareto front*. Once learned, the Pareto front stays fixed, since it does not depend on the preferences of the decision maker. The decision maker can then use the Pareto front to review all available policies, and use this knowledge to select their preferred one [19]. This greatly simplifies the task of the decision maker, as the consequences of any given choice are clear and explainable in advance. Additionally, if the decision maker changes their mind at a later time, they can simply select a different policy without the need to re-tune an optimization metric and perform the learning process again.

In this work we propose a novel method, *Pareto Conditioned Networks* (PCN), that is able to efficiently learn the policies that belong to the Pareto front. PCN conditions a single neural network on the desired compromise, so that it outputs the policy predicted to achieve it. Our method is sample-efficient, as it feeds the experience used for different compromises into the same network, thus allowing to share experience across policies. This also means it does not need to learn a policy independently for each trade-off found in the Pareto front, which is an approach taken by several works in multi-objective reinforcement learning (MORL) [17, 20]. Moreover, we make minimal assumptions concerning the utility function — i.e., the range of possible preferences of the decision maker — as opposed to the often-used assumption of linear scalarization of the objectives in the MORL literature [1, 25]. Finally, our method is scalable with respect to the number of objectives, as opposed to many of the current state-of-the-art MORL methods, who often limit themselves to 2- or 3-objective problems [8].

2 BACKGROUND

2.1 Multi-Objective Reinforcement Learning

In *reinforcement learning* (RL), an agent learns to optimise its behaviour by interacting with the environment. In this paper, we deal with decision problems with multiple objectives, and model this as a *multi-objective Markov decision process* (MOMDP). A MOMDP is a

tuple, $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, \vec{R}, n \rangle$, where $\mathcal{S}, \mathcal{A}$ are the state and action spaces respectively, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is a probabilistic transition function, γ is a discount factor determining the importance of future rewards and $\vec{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^n$ is an n -dimensional vector-valued immediate reward function, with n being the number of objectives of the problem. In single-objective RL, $n = 1$ while in *multi-objective reinforcement learning (MORL)*, $n > 1$.

When $n = 1$, the goal is to find the policy π^* that maximizes the expected sum of discounted rewards:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 \right] \quad (1)$$

In contrast, when $n > 1$, this sum can lead to returns for which, without any additional information, there is no clear winner (e.g., we cannot decide which return is optimal between $(0, 10)$, $(5, 5)$ or $(10, 0)$). A solution with values for all objectives lower than another is said to be *dominated*. A solution for which it is impossible to improve one of the objectives without hampering another (e.g., any of the solutions in the previous example) is said to be *Pareto-efficient*. The set of all Pareto-efficient solutions — i.e., the set of all best possible compromises — is called the *Pareto front*.

We can represent the decision maker preferences with a *utility function* $u : \mathbb{R}^n \rightarrow \mathbb{R}$, which converts the multi-objective returns back to a single scalar signal. Unfortunately, in most settings, the utility function is not known in advance: it either varies in each individual instance of the problem, or it cannot be easily formalized by the decision maker. Thus, in this work we assume that u is unknown but monotonically increasing, and we focus on learning the full set of non-dominated solutions represented by the Pareto front. In addition, we must learn multiple separate policies that can reach all of the non-dominated solutions, since without knowing u we must assume that any solution could be optimal. After learning, the decision maker can select their preferred solution and execute the corresponding policy.

We now define these concepts more formally. A policy π is said to Pareto-dominate another policy π' if its expected return $\mathbf{V}^{\pi}$, also called V -value, is higher or equal *across all objectives* than $\mathbf{V}^{\pi'}$, and there exist at least an objective where $\mathbf{V}^{\pi}$ is better than $\mathbf{V}^{\pi'}$:

$$\mathbf{V}^{\pi} \succ_P \mathbf{V}^{\pi'} \iff (\forall i : \mathbf{V}_i^{\pi} \geq \mathbf{V}_i^{\pi'}) \wedge (\exists i : \mathbf{V}_i^{\pi} > \mathbf{V}_i^{\pi'}) \quad (2)$$

where $\succ_P$ is the Pareto-dominance operator.

Our goal is then to find the set of policies that are not dominated by any other policy:

$$\Pi^* = \{ \pi \in \Pi \mid \nexists \pi' \in \Pi : \mathbf{V}^{\pi'} \succ_P \mathbf{V}^{\pi} \} \quad (3)$$

where Π is the set of all possible policies. Π^* then maps to the Pareto front $\mathcal{F} = \{ \mathbf{V}^{\pi} \mid \pi \in \Pi^* \}$. In general, we call any set of V -values mapped from a set of policies a *solution set*, and any solution set composed only of non-dominated V -values a *coverage set*. Thus, the Pareto front is the optimal coverage set of the problem.

Learning the full set of Pareto-efficient policies Π^* requires that the policies $\pi^* \in \Pi^*$ are deterministic stationary policies [19]. This is useful in settings where stochastic policies are not desired, such as the management of a hydroelectric power plant. In that scenario, the decision maker does not want to be presented with a policy that has a probability of completely draining the water reservoir even if

that policy is optimal, as it would have catastrophic consequences for nearby towns [8].

2.2 Multi-Objective Metrics

Comparing the learned coverage sets of different algorithms is a non-trivial task, as one algorithm's output might dominate the other in some part of the objective-space, but be dominated in another. Intuitively, one would generally prefer the algorithm that obtains better returns for a wider range of utility functions. In this work we use several metrics to evaluate an algorithm's performance.

The most widely used metric in the literature is called the *hypervolume* [27]. This metric evaluates the learned coverage set by computing its volume w.r.t. a fixed specified reference point. The reference point is taken as a lower bound on the achievable returns so that the volumes are always positive. Thus, the hypervolume envelops all possible V -values that are dominated by that coverage set, with more dominating coverage sets having a larger hypervolume. This metric is by definition the highest for the Pareto front, as no other possible solution can increase its volume (since they are all dominated). While the hypervolume metric is widely used and does give a measure of the coverage of a solution set, it can be difficult to interpret. The benefit of a certain increase or decrease in hypervolume is not readily apparent to the end user, and does not necessarily correlate to significant changes in expected utility. When working in high-dimensional objective-spaces, adding or removing a single point can lead to wildly different hypervolume values, especially if the point lies close to an extremum of the space.

To combat these limitations, we additionally evaluate our work with a different metric called the ϵ -indicator I_{ϵ} [28]. I_{ϵ} measures how close a coverage set is to the Pareto front $\mathcal{F}$. The ϵ -indicator of a coverage set $\hat{\Pi}$ is computed such that, for every $\mathbf{V}^{\pi}$ of the Pareto front, there exist a V -value in the coverage set that is at most ϵ smaller than $\mathbf{V}^{\pi}$:

$$I_{\epsilon} = \inf_{\epsilon \in \mathbb{R}} \{ \forall \mathbf{V}^{\pi} \in \mathcal{F}, \exists \mathbf{V}^{\pi'} \in \hat{\Pi} : \|\mathbf{V}^{\pi} - \mathbf{V}^{\pi'}\|_{\infty} \leq \epsilon \} \quad (4)$$

From a user's perspective, I_{ϵ} has the intuitive meaning of showing that the proposed coverage set is at most ϵ worse than any V -value of the Pareto front. The main disadvantage of this metric is that to compute it we need the true Pareto front, so it can only be used in test problems with a known structure.

While the ϵ -indicator has nice theoretical properties guaranteeing performance in the worst case [26] — i.e., the maximum utility loss (MUL) with respect to any possible utility function — it does not give any information about the expected utility loss (EUL). This makes it a highly pessimistic metric; the ϵ -indicator will still report bad performance even if nearly the whole Pareto front is learned exactly, as long as a single point is not correctly modeled.

While measuring the expected utility loss precisely requires knowing in advance the distribution of possible utility functions, we propose a variation on the I_{ϵ} metric that aims to approximate an upper bound to this value, by making an additional assumption.

In particular, we propose computing $I_{\epsilon-\text{mean}}$, that assumes that each point in the Pareto front is equally likely to be selected as the best choice by a randomly sampled utility function. We compute $I_{\epsilon-\text{mean}}$ by taking the mean of the computed ϵ values, which are computed in the same way as the original I_{ϵ} metric. Because the

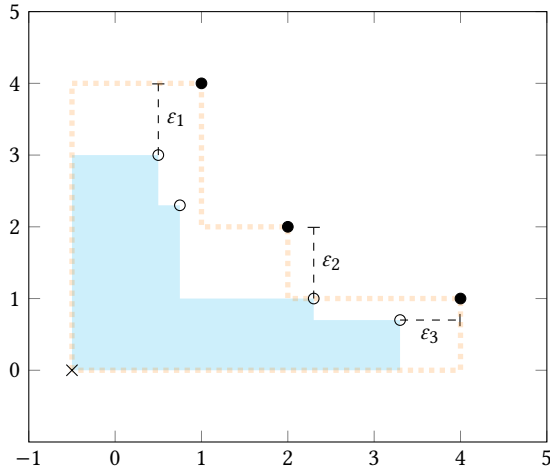


Figure 1: Pareto front and coverage set in 2-objective environment. The Pareto front is the set containing the best compromises that can be achieved (black dots). We want to learn a coverage set that is as close as possible to the true Pareto front (white dots). The hypervolume metric, in light blue, measures the volume of all dominated solutions w.r.t. some reference point (cross). The ϵ metrics first compute the maximum distance between each point in the Pareto front and its closest point in the coverage set (ϵ_i). We can then take their maximum value to compute the I_ϵ metric, or their mean value to obtain the $I_{\epsilon-mean}$ metric of the coverage set.

maximal utility loss of incorrectly preferring another vector in the coverage set over a given vector in the Pareto front is $\epsilon\sqrt{n}L$ [26], where L is the Lipschitz constant that described the level of continuity of the utility function, the $I_{\epsilon-mean}$ metric describes an upper bound on the EUL, given that each vector in the Pareto set is (approximately) equally likely to be the vector that maximises the user’s utility.

Figure 1 shows a visual representation of the hypervolume and ϵ metrics in 2 dimensions.

2.3 Reward Conditioned Policies

Our work is inspired by the Reward Conditioned Policies algorithm proposed by [11, 21]. Using neural networks as function approximators in RL comes with many challenges. One of them is that the target (e.g., the optimal action of the policy) is not known in advance — as opposed to classical supervised learning where the ground-truth target is provided. As the behavior of the agent improves over time, the action used as target can change, often leading to hard-to-tune and brittle learners [7, 13].

Instead of trying to continuously improve the policy by learning actions that should lead to the highest cumulative reward, Reward Conditioned Policies flips the problem, by learning actions that should lead to any *desired* cumulative reward (be it high or low). In this way, all past trajectories can be reused for supervision, since their returns are known, as well as the actions needed to reach said returns. We can thus train a policy that, conditioned on a desired return, provides the optimal action to reach said return. By

leveraging the generalization properties of neural networks, we can accumulate incrementally better experience by conditioning on increasingly higher reward-goals.

In our work, we condition our policy on a multi-objective return, such that we can execute policies to reach diverse points on the Pareto front. We propose a training regimen focused on increasing the current solution set uniformly across the whole objective-space to avoiding catastrophic forgetting.

3 RELATED WORK

Current MORL methods can be broadly divided into two main categories: *single-policy* and *multi-policy* algorithms [23]. In the first case, one tries to learn a single, optimal policy for a given set of preferences, i.e., for a known utility function. In the second case, the utility function is unknown (or uncertain) and the goal is to learn a set of policies that cover all possible utility functions, i.e., a coverage set.

Most of the recent work on MORL falls into the second category and assumes an unknown, but linear utility function. However, this restricts the solution set that can be learnt, as linear scalarization assumes the Pareto front to be convex. In the linear setting, the goal is to train an agent such that the optimal policy can be recovered for any preference weights. [20] propose Optimistic Linear Support (OLS), a generic method that iteratively selects different sets of weights and calls a single-objective learner as subroutine to find the corresponding optimal policy. [14] extend this method for Deep RL. Another approach, taken by [2, 9], is to directly optimise on the coverage set without single-objective subroutine, by modifying the Bellman equation. Contrary to PCN, these methods are unable to discover any V-values on the concave regions of the Pareto front [5].

Using conditioned networks has been explored in MORL, but again restricted to the linear scalarization setting. In [3], Fitted Q-Iteration (FQI) is extended to use a modified Q-network conditioned on *preference weights* instead of target returns. Similarly, [1] use such a conditioned Q-network to extend Deep Q-Networks (DQN). Moreover, a similar network is used in [25], in combination with a multi-objective Bellman operator.

Our work also can be related to imitation learning, as it also uses supervised learning to learn a policy [15, 22]. However, imitation learning requires expert trajectories to train on, while PCN generates its own set of trajectories.

When the utility function can be any monotonically-increasing function, [24] adapt tabular Q-learning to directly learn the Pareto front. However, it is limited to discrete low-dimensional state-spaces.

Finally, [17] learn to reach the Pareto front using a modified policy gradient search, and [16] do this by modifying evolution strategies. Both algorithms make the same assumptions on the utility functions as PCN and are used as baselines in our experimental section (Section 5).

4 PARETO CONDITIONED NETWORKS

In this Section we introduce our main contribution, the Pareto Conditioned Networks (PCN) algorithm. The key idea behind our approach is to use supervised learning techniques to improve the

policy instead of resorting to temporal-difference learning. As explained in Section 2.3, this eliminates the moving-target problem, resulting in stable learning.

PCN uses a single neural network that takes a tuple $\langle s, \hat{h}, \hat{R} \rangle$ as input. They represent, for state s , the return $\hat{R}$ that PCN should reach at the end of the episode, i.e. the *desired return* of the decision maker. The *desired horizon* $\hat{h}$, that says how many timesteps should be executed before reaching $\hat{R}$. At execution time, both $\hat{h}$ and $\hat{R}$ are chosen by the decision maker at the start of the episode.

PCN’s neural network has a separate output for each action $a_i \in \mathcal{A}$. Each output represents the confidence the network has that, by taking the corresponding action, the desired return will be reached in the desired number of timesteps. We can draw an analogy with a classification problem where the network should learn to classify $\langle s, \hat{h}, \hat{R} \rangle$ to its corresponding label a_i .

Similarly as with classification, PCN requires a labeled dataset with training examples to learn a mapping from input to label. Contrary to classification, however, the data present in the dataset is not fixed. PCN collects data from the trajectories experienced while exploring the environment (see Section 4.1). Thus, the dataset improves over time, as we collect better and better trajectories. In particular, since the ability of PCN to reach Pareto-dominating solutions depends on the data on which its network is trained, we want to keep only relevant experiences. We do so by limiting the size of the dataset and pruning it in such a way so to keep only tuples with $\hat{R}$ ’s from different parts of the objective-space (see Section 4.4).

We collect new data for several episodes, after which we re-train the network with a number of batch updates from the new dataset (see Section 4.2). This improves the policies induced by the network, which in turn allows to gather better data for the next training batch.

4.1 Building the dataset

As mentioned before, each datapoint in PCN’s dataset is comprised of an input $\langle s, \hat{h}, \hat{R} \rangle$ and an output a . These are computed from observed transitions in the environment. As the dataset is empty at first, we execute a random policy on the environment for the first few episodes in order to collect a variety of trajectories.

After each episode is completed we store its trajectory. Then, for each timestep t of the trajectory, we know how many timesteps are left until the end is reached, i.e., the episode’s horizon $h_t = T - t$. We can also compute the cumulative reward obtained from timestep t onward, i.e., $R_t = \sum_{i=t}^T \gamma^i r_i$. Since for this trajectory executing action a_t in state s_t resulted in return R_t in h_t timesteps, we add a datapoint with input $\langle s, \hat{h}, \hat{R} \rangle = \langle s_t, h_t, R_t \rangle$ and output $a = a_t$ to the dataset. In other words, when the observed return corresponds to the desired return in that state, then a_t is the optimal action to take. Figure 2 shows how a full trajectory is decomposed into individual datapoints.

4.2 Training the Network

PCN’s network architecture uses a separate embedding for the state and another one for the desired return and horizon. The desired return and horizon are concatenated together and multiplied by a scaling factor to normalize their values. They then pass through a

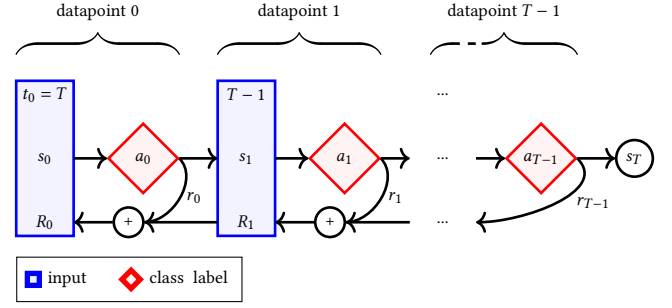


Figure 2: Conversion from a trajectory to labeled datapoints. For each timestep, we extract a single datapoint. The input (blue) is composed of the state at that timestep, and total return and number of timesteps until the end of the episode. The label (red) is the action taken at that timestep.

single fully connected layer followed by a sigmoid function. Similarly, the state-embedding also ends with a fully connected layer and sigmoid activation. Both layers have the same number of output nodes (we use 64 for all our experiments), which are combined together using the Hadamard product. The Hadamard product has been shown to make a more effective use of conditioning variables [18], and the sigmoid used on both outputs ensure that both embeddings are equally important. Finally, the resulting output passes through a multilayer perceptron that has a separate output node for each action, and a single hidden layer of 64 nodes with a ReLU activation function.

PCN trains the network as a classification problem, where each class represents a different action. Transitions $x = \langle s_t, h_t, R_t \rangle$, $y = a_t$ are sampled from the dataset, and the ground-truth output y is compared with the predicted output $\hat{y} = \pi(s_t, h_t, R_t)$. The predictor (i.e., the policy) is then updated using the cross-entropy loss function:

$$H = - \sum_{a \in \mathcal{A}} y_a \log \pi(a | s_t, h_t, R_t) \quad (5)$$

where $y_a = 1$ if $a = a_t$ and $y_a = 0$ otherwise.

The network is re-trained periodically, but only after a set number of episodes (which is a hyperparameter that depends on the problem), to ensure that enough new experience has been collected and that the underlying dataset has been improved sufficiently.

4.3 Policy Exploration

As our dataset is composed of transitions collected from training experience, we can see that the quality of our dataset crucially depends on the quality of the executed trajectories. It is unrealistic to expect PCN to reliably produce trajectories with high-valued desired returns when it has only been trained on datapoints originating from random trajectories. Rather, we can expect PCN to produce trajectories with returns in the range of the ones from the current training data. Therefore, if we obtain trajectories reaching high returns, PCN will be able to confidently return high-return policies.

PCN leverages the fact that, due to the generalization capabilities of neural networks, the policies obtained from the network will still be reliable even if the desired return is marginally higher than what

is present in the training data. In fact, they will perform similar actions to those in the training data, but lead to a higher return. Thus, we incrementally condition the network on better and better returns, in order to obtain trajectories that extend the boundaries of PCN’s current coverage set.

More precisely, we randomly select a non-dominated return $\mathbf{R}_{nd}$ and its corresponding horizon $\hat{h}$ from the dataset. By randomly picking a non-dominated return from the entire coverage set we ensure equal chance of improvement to each part of the objective space. However, using $\mathbf{R}_{nd}$ exactly would induce the network to only replicate the already observed sampled trajectory so, as a second step, we choose a single objective o to improve upon. We then increase the desired value for that objective to obtain a new target return. PCN determines the magnitude of the increase by computing the standard deviation σ_o for the selected objective, using all non-dominated returns from the trajectories in the dataset. The magnitude is then sampled from the uniform distribution $U(0, \sigma_o)$ and added to $R_{nd,o}$ to form our desired return $\hat{\mathbf{R}}$. By restricting the improvement to at most σ_o , $\hat{\mathbf{R}}$ stays in the range of possible achievable returns and, by only modifying one objective at a time, the changes to the network’s input compared to the training data are kept at a minimum.

With $\hat{\mathbf{R}}$ and $\hat{h}$ selected, PCN can condition its network and act during the training episode. At the start of the episode, $\langle s_0, \hat{h}, \hat{\mathbf{R}} \rangle$ results in executing a_0 and observing $\mathbf{r}_0, s_1$. PCN then updates the desired return and horizon such that they stay consistent throughout the episode: $\hat{\mathbf{R}} \leftarrow \hat{\mathbf{R}} - \mathbf{r}_0$ and $\hat{h} \leftarrow \max(\hat{h} - 1, 1)$. We ensure that $\hat{h}$ is at least 1 to avoid impossible desired horizons. PCN can then choose an action for s_1 . This process is repeated until PCN encounters a terminal state. The trajectory is then added to the dataset, using the conversion to datapoints explained in Section 4.1.

To increase the range of observations in the environment during training, PCN samples actions from a categorical distribution with each action’s confidence score corresponding to its probability of being sampled. This is done by using a softmax function on the network’s output. Please note that at execution time — i.e., after the training process — we use a deterministic policy by systematically selecting the action with the highest confidence. This is because, as mentioned in Section 2, only deterministic policies are optimal when learning the complete set of Pareto-efficient policies.

4.4 Updating the Dataset

As it collects new experience, PCN needs to use it to train its network without forgetting about previous, relevant experience. Unfortunately, measuring relevance of data in our setting is non-trivial. We mainly care about the non-dominated solutions, since those are the ones that compose our coverage set, so ideally we would only keep their associated trajectories in our dataset. However, focusing solely on the current coverage set can lead to performance degradation if it is composed of too few V-values. This is because, during exploration (see Section 4.3), we will keep collecting very similar trajectories to the ones we already have. In turn, this will reinforce PCN’s strategy to focus only on these few policies, disrupting the learning process. Thus, throughout training we keep a dataset of N trajectories, favoring trajectories that span different parts of the objective-space while removing highly clustered solutions. To do

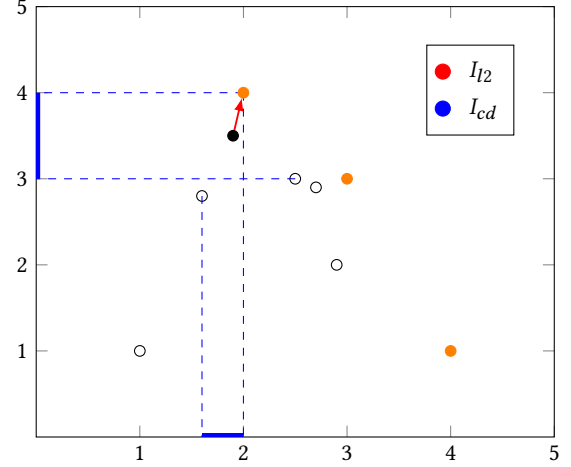


Figure 3: The I_{ds} metric for the solid black dot combines its negative L2-norm distance (red arrow) to its closest non-dominated neighbor (orange), and its crowding distance as the sum, for each dimension, of the max distance between a point’s upper and lower neighbour (blue).

this we employ an additional metric, the *crowding distance*, that assigns a lower score to points with close neighbors. We then combine both the distance to the coverage set and the crowding distance in a single metric, which we use to prune less relevant points from the dataset.

We measure our preference for non-dominated V-values by computing for each solution its negative L2-norm distance, I_{l2} , to its closest non-dominated solution in the dataset. Non-dominated V-values thus score the highest with $I_{l2} = 0$.

$$I_{l2,i} = -\min \|p_i - p_j\|_2, p_j \in \hat{\Pi} \quad (6)$$

where i is the index of the i -th solution in the dataset and p_j is a non-dominated point in the current coverage set $\hat{\Pi}$.

We measure the level of clustering of a V-value, I_{cd} , using the crowding distance [6]. It assigns a score for each solution based on the distance between its neighbours in each dimension. Thus, V-values with close neighbors will have lower scores, while more isolated points will have higher scores. Algorithm 1 shows how the crowding distance is computed in practice.

Algorithm 1 Crowding Distance

Input: p points

Output: The crowding distance I_{cd}

```

1: for  $o \leftarrow 0$  to  $n$  do
2:    $s_i \leftarrow \arg \text{sort } p_{:,o}$ 
3:   for  $j \leftarrow 0$  to  $\text{len}(p)$  do
4:      $n_u, n_d \leftarrow s_{i_{j+1}}, s_{i_{j-1}}$ 
5:      $c_{s_{ij},o} = p_{n_u} - p_{n_d}$ 
6:   end for
7: end for
8: return  $I_{cd} \leftarrow \sum_o c_{:,o}$ 
```

We then combine I_{l2} with I_{cd} in a single score metric, which we call *dominating score*, I_{ds} . We define it as:

$$I_{ds,i} = \begin{cases} I_{l2,i} & \text{if } I_{cd,i} > 0.2 \\ 2(I_{l2,i} - c) & \text{if } I_{cd,i} \leq 0.2 \end{cases} \quad (7)$$

Note that since I_{l2} corresponds to a negative distance, if a point is crowded — i.e., $I_{cd,i} \leq 0.2$ — we double the distance penalty. In addition, we add an additional small penalty c to crowded points to prune, to detect and prune duplicate points on the coverage set. Figure 3 shows an example on how to compute the L2-norm and crowding distances for a given solution set.

5 EXPERIMENTS

Most state-of-the-art algorithms in MORL assume that the utility function can be expressed as a linear scalarization. This makes them unsuitable as relevant baselines, as their setting is different from ours. Thus, for all experiments, we compare PCN with 2 baselines that share our same assumptions on the utility function and that learn a set of policies that estimates the whole Pareto front. The first baseline, Multi-Objective Natural Evolution Strategies (MONES) [16] uses a parametrized policy. It learns a distribution over the policy parameters such that sampling from this distribution produces a Pareto-efficient policy. Different samples produce different Pareto-efficient policies, each leading to a different non-dominated return. The advantage of this method is that it can produce an infinite number of different policies. However, the main drawback is that we do not know the return of the sampled policy without executing it first.

The second baseline used is the Radial Algorithm (RA) [17]. RA trains a fixed number of independent policies. Each policy is trained using a policy gradient algorithm, where the gradients w.r.t. the different objectives are weighted together. Using different weights on these gradients produces distinct policies, each aiming for different regions of the objective-space. The main disadvantage of this approach is that every new policy is learnt independently, disregarding potentially useful experience encountered by other policies.

In contrast, our approach makes efficient usage of encountered experience as it learns a single network that, when conditioned on a desired return, produces a policy with predictable behavior.

When not mentioned otherwise, all results are averaged over 5 runs.

The results for all experiments are summarized in Table 1. Figures 4-6 show, for each environment, the coverage set found by each algorithm, with dominated solutions filtered out.

We experimentally validate our method on three multi-objective benchmarks:

- Deep-Sea-Treasure [23], a well-known 2-objective grid-world problem (see Section 5.1),
- Minecart [1], a 3-objective problem with a continuous state-space (see Section 5.2),
- Crossroad, a novel 2-objective traffic environment, with high-dimensional pixel-like states (see Section 5.3).

In addition, in Section 6 we propose an additional novel high-objective environment, Walkroom, where we test our algorithms with up to 9 objectives.

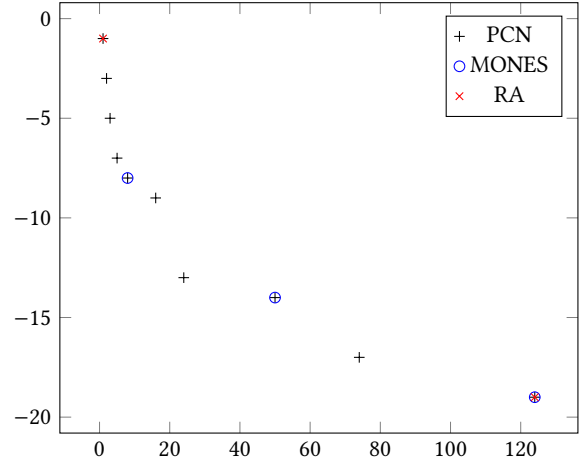


Figure 4: Best coverage sets for each algorithm in the Deep Sea Treasure environment. PCN is the only algorithm that recovered the full Pareto front.

5.1 Deep-Sea-Treasure

Deep-Sea-Treasure (DST) is a well known environment in multi-objective literature [23], where the agent controls a submarine in search for treasure hidden in the depth of the ocean. The agent must balance a trade-off between fuel consumption and treasure value. Navigating consumes fuel, but deeper treasures are worth more than shallow ones.

DST is a fairly small environment, which allows us to compute Pareto front analytically. It is composed of 10 different points, which form a concave front.

Figure 4 shows the discovered points of the Pareto front by PCN and our two baselines. Only PCN is able to fully recover the Pareto front. RA only discovers the extrema, and is unable to find any of the points in the concave part of the front. While MONES performs slightly better, the number and value of points it discovered was highly variable depending on the run, explaining the high variance seen in Table 1.

5.2 Minecart

Minecart is a complex environment with a continuous state-space [1]. Starting at a base station, the agent controls a cart with the goal to extract ores from mines scattered in the environment, and sell them back at its base.

The agent can execute 6 possible actions: it can accelerate, decelerate and rotate the cart to the left or right. It can mine ores, which will only be effective if it is located in a mine. It can also simply do nothing. There are 2 types of ores located in the different mines, where each type represents a separate objective. The cart has a limited capacity, so it cannot be filled indefinitely. Finally, actions consume fuel, making the Minecart problem a 3-objective problem.

Because the cart has a limited capacity to store ores, the agent must decide the ratio of each ore present in the cart. This is why, as can be seen in Figure 5, the vast majority of policies discovered by PCN are laid out in a straight line: the sum of R_0 and R_1 equals 1.5 (which is the cart capacity). A few policies only partially fill the

	hypervolume			I_ε indicator		
	PCN (ours)	MONES	RA	PCN (ours)	MONES	RA
DST	22845.40 $\pm$ 19.20*	17384.83 $\pm$ 6521.10	22437.40 $\pm$ 49.20	0.039 $\pm$ 0.087*	0.687 $\pm$ 0.222	0.667 $\pm$ 0.000
Minecart	197.56 $\pm$ 0.70*	123.81 $\pm$ 23.03	123.92 $\pm$ 0.25	0.271 $\pm$ 0.087*	1.596 $\pm$ 0.889	1.000 $\pm$ 0.000
Crossroad	539.53 $\pm$ 6.27*	429.09 $\pm$ 27.47	466.02 $\pm$ 31.23	0.247 $\pm$ 0.172*	0.660 $\pm$ 0.200	0.408 $\pm$ 0.039

Table 1: Mean and standard deviation of hypervolume and ε -indicator across all 5 runs for all algorithms. For hypervolume, higher is better. For I_ε , lower is better. Best results are highlighted in bold and with an asterisk.

cart, saving a bit of fuel in the process. In comparison, the coverage sets discovered by both baselines only contain a small subset of the possible ore ratios, and they systematically consume more fuel than our method. Finally, Table 1 reports that PCN achieves a low variance in hypervolume across the different runs, which shows that PCN is consistent in finding these diverse and efficient policies.

5.3 Crossroad

We evaluate our proposed method on a novel traffic environment, Crossroad, developed using the SUMO framework [12]. In this environment, the agent controls the traffic lights at a busy intersection between two bidirectional roads, an horizontal one with two lanes and a vertical one with a single lane. The agent can choose between 2 actions, either switching the lights or not. We consider 2 objectives: the first is the traffic flow, computed as the number of cars that leave the intersection. The second objective is the car waiting time, i.e., the number of timesteps a car has to wait before exiting the crossroad. Thus, favoring traffic on the horizontal road will favor the first objective, while alternating often will favor the second.

Figure 6 shows the coverage sets found by PCN and the two baselines. The points discovered by our method dominate the ones found by the baselines across nearly the whole objective-space. There is a single exception is the rightmost extremum, which corresponds to the policy that never switches the light, only allowing cars from the major road to cross the intersection. This policy is pretty simple, as it consists of always performing the same action, which explains why all methods find it.

6 SCALING UP THE OBJECTIVE-SPACE

Our experimental section shows that our proposed method significantly outperforms the baselines in several settings, with discrete, continuous and high-dimensional state-spaces. It also empirically shows that the coverage sets discovered by PCN contain more non-dominated solutions than our selected baselines. Nevertheless, the vast majority of benchmarks used in MORL, including the ones used in our experiments, are limited to 2, sometimes 3 different objectives [8].

To get a better understanding of the performance of PCN w.r.t. the number of objectives, we devise a synthetic environment, *Walkroom*, that can be instanced with an arbitrary number of objectives. Walkroom takes inspiration from Deep Sea Treasure, and is modeled as an n -dimensional grid-world in which the agent can move in every cardinal direction. Thus, the dimensions correspond to the number of objectives of the environment. The action space increases linearly with the number of objectives ($|\mathcal{A}| = 2n$). At each timestep,

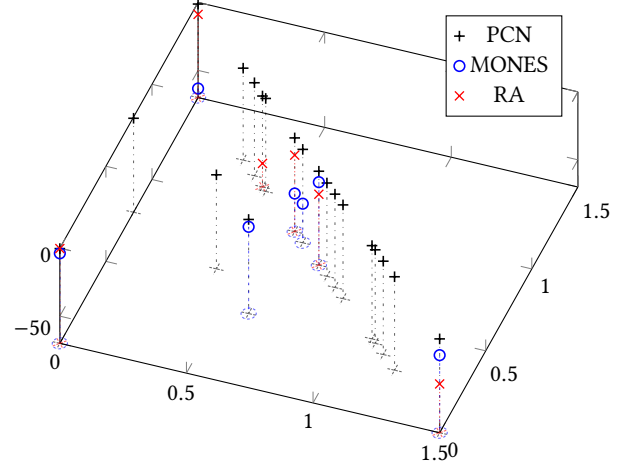


Figure 5: Best coverage sets for each algorithm in the Minecart environment. The straight shape of PCN’s coverage set is due to the cart weight limit. PCN’s coverage set fully dominates the ones from the baselines.

the agent receives a -1 reward for the objective corresponding to its moving dimension, and a 0 reward for all other objectives. There are no other rewards. Similarly to Deep Sea Treasure, Walkroom has a set of goal states positioned along an uneven border, so that reaching each goal state results in a different Pareto-efficient solution. The optimal policies are thus to go directly towards any of the border-positions, at which point the episode ends.

We evaluate each method in a set of randomly generated Walkroom environments, from 2 to 9 dimensions. We perform 20 runs for every algorithm, on every version of the environment ($n = 2, \dots, 9$). Note that we do not plot the discovered coverage sets since this is not possible for $n > 3$. Instead, we show boxplots of the hypervolume and ε metrics computed on all runs. Since the Pareto fronts can be computed analytically when the environments are generated, the ε metrics give us an accurate representation of the coverage of each solution set found by each learning algorithm.

Results are summarized in Figure 7. RA performs poorly compared to the other algorithms across all metrics. This is because the number of policies that RA requires to cover the objective-space increases exponentially with the number of objectives. Thus, training time must be split between more and more policies, which reduces their individual performance. For the same reason, we were unfortunately unable to compare in RA in environments with more than 5 objectives, due to its exponential computational costs.

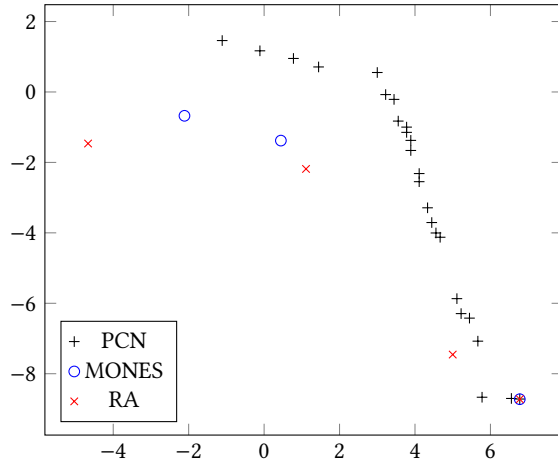


Figure 6: Best coverage sets for each algorithm in the Cross-road environment. PCN fully dominates the baselines but for the naive policy that never switches the traffic lights (bottom-right point).

For $n \leq 4$, MONES achieves similar or better scores than PCN in the I_ϵ metric. This is likely due to PCN missing some points from the Pareto front, which impact this metric significantly. However, we can see from the hypervolume and $I_{\epsilon-mean}$ metrics that PCN generally performs better than MONES on the rest of the Pareto front. Surprisingly, for $n > 4$ MONES performs significantly worse than PCN. This might be because the parameter distributions of MONES are not able to explore efficiently in very large dimensional spaces.

7 CONCLUSION

We have presented a novel algorithm, Pareto Conditioned Networks, which is able to efficiently and effectively learn coverage sets in multi-objective sequential problems. PCN uses a single neural network to generalize experience across all possible multi-objective returns, learning coverage sets even in concave Pareto fronts.

We evaluated the empirical performance of PCN in several environments against state-of-the-art benchmarks. PCN was able to consistently obtain higher returns than the baselines throughout the whole objective-space, demonstrating its ability to exhaustively discover optimal coverage sets. In addition, PCN demonstrated its ability to learn the Pareto front even when dealing with a large number of objectives.

While PCN can work in continuous state-spaces, its network architecture is currently limited to discrete action-spaces. However, the main ideas behind PCN do not change for continuous action-spaces and PCN can be extended to continuous actions by changing the classification problem to a regression one. We leave this for future work.

ACKNOWLEDGMENTS

The authors would like to acknowledge FWO (Fonds Wetenschappelijk Onderzoek) for their support through the SB grant of Eugenio Bargiacchi (#1SA2820N). This research was additionally supported

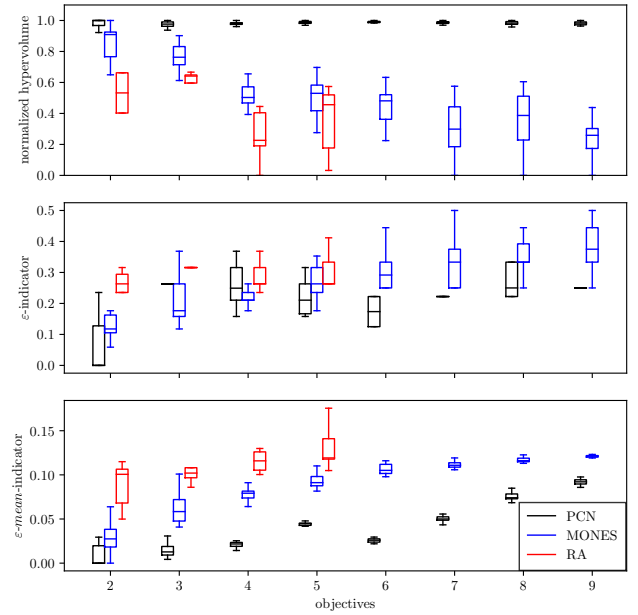


Figure 7: Box plots of normalized hypervolume, I_ϵ and $I_{\epsilon-mean}$ in the Walkroom environments, from 2 to 9 objectives. We normalized the hypervolumes as their true values scale exponentially with the number of objectives (for $n = 9$, in the order of 10^9). For hypervolume, higher is better. For ϵ -indicator metrics, lower is better.

by funding from the Flemish Government under the “Onderzoek-sprogramma Artificialle Intelligentie (AI) Vlaanderen” programme.

We would also like to thank Diederik M. Roijers for helpful feedback.

REFERENCES

- [1] Axel Abels, Diederik Marijn Roijers, Tom Lenaerts, Ann Nowé, and Denis Steckelmacher. 2019. Dynamic Weights in Multi-Objective Deep Reinforcement Learning. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, Long Beach, California, USA, 11–20.
- [2] Leon Barrett and Srinivas Narayanan. 2008. Learning all optimal policies with multiple criteria. In *Proceedings of the 25th international conference on Machine learning*. ACM, 41–47.
- [3] Andrea Castelletti, Francesca Pianosi, and Marcello Restelli. 2012. Tree-based fitted Q-iteration for multi-objective Markov decision problems. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [4] A Castelletti, Francesca Pianosi, and Marcello Restelli. 2013. A multiobjective reinforcement learning approach to water resources systems operation: Pareto frontier approximation in a single run. *Water Resources Research* 49, 6 (2013), 3476–3486.
- [5] Indraneel Das and John E Dennis. 1997. A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. *Structural optimization* 14, 1 (1997), 63–69.
- [6] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, and Tanaka Meyarivan. 2000. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In *International conference on parallel problem solving from nature*. Springer, 849–858.
- [7] Justin Fu, Aviral Kumar, Matthew Soh, and Sergey Levine. 2019. Diagnosing bottlenecks in deep q-learning algorithms. In *International Conference on Machine Learning*. PMLR, 2021–2030.
- [8] Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M Zintgraf, Richard

- Dazeley, Fredrik Heintz, et al. 2021. A practical guide to multi-objective reinforcement learning and planning. *arXiv preprint arXiv:2103.09568* (2021).
- [9] Kazuyuki Hiraoka, Manabu Yoshida, and Taketoshi Mishima. 2009. Parallel reinforcement learning for weighted multi-criteria model with adaptive margin. *Cognitive neurodynamics* 3, 1 (2009), 17–24.
- [10] Ammar Jalalimanesh, Hamidreza Shahabi Haghighi, Abbas Ahmadi, Hossein Hejazian, and Madjid Soltani. 2017. Multi-objective optimization of radiotherapy: distributed Q-learning and agent-based simulation. *Journal of Experimental & theoretical artificial intelligence* 29, 5 (2017), 1071–1086.
- [11] Aviral Kumar, Xue Bin Peng, and Sergey Levine. 2019. Reward-conditioned policies. *arXiv preprint arXiv:1912.13465* (2019).
- [12] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. 2018. Microscopic Traffic Simulation using SUMO, In The 21st IEEE International Conference on Intelligent Transportation Systems. *IEEE Intelligent Transportation Systems Conference (ITSC)*. <https://elib.dlr.de/124092/>
- [13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529.
- [14] Hossam Mossalam, Yannis M. Assael, Diederik M. Roijers, and Shimon Whiteson. 2016. Multi-Objective Deep Reinforcement Learning. *CoRR* abs/1610.02707 (2016). [arXiv:1610.02707](https://arxiv.org/abs/1610.02707) <http://arxiv.org/abs/1610.02707>
- [15] Takayuki Osa, Joni Pajarinen, Gerhard Neumann, J Andrew Bagnell, Pieter Abbeel, and Jan Peters. 2018. An algorithmic perspective on imitation learning. *arXiv preprint arXiv:1811.06711* (2018).
- [16] Simone Parisi, Matteo Pirota, and Jan Peters. 2017. Manifold-based multi-objective policy search with sample reuse. *Neurocomputing* 263 (2017), 3–14.
- [17] Simone Parisi, Matteo Pirota, Nicola Smacchia, Luca Bascetta, and Marcello Restelli. 2014. Policy gradient approaches for multi-objective sequential decision making. In *2014 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2323–2330.
- [18] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. 2018. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [19] Diederik M Roijers, Peter Vamplew, Shimon Whiteson, and Richard Dazeley. 2013. A survey of multi-objective sequential decision-making. *Journal of Artificial Intelligence Research* 48 (2013), 67–113.
- [20] Diederik Marijn Roijers, Shimon Whiteson, and Frans A Oliehoek. 2015. Computing convex coverage sets for faster multi-objective coordination. *Journal of Artificial Intelligence Research* 52 (2015), 399–443.
- [21] Juergen Schmidhuber. 2019. Reinforcement Learning Upside Down: Don't Predict Rewards—Just Map Them to Actions. *arXiv preprint arXiv:1912.02875* (2019).
- [22] Wen Sun, J Andrew Bagnell, and Byron Boots. 2018. Truncated horizon policy search: Combining reinforcement learning & imitation learning. *arXiv preprint arXiv:1805.11240* (2018).
- [23] Peter Vamplew, Richard Dazeley, Adam Berry, Rustam Issabekov, and Evan Dekker. 2011. Empirical evaluation methods for multiobjective reinforcement learning algorithms. *Machine learning* 84, 1-2 (2011), 51–80.
- [24] Kristof Van Moffaert and Ann Nowé. 2014. Multi-objective reinforcement learning using sets of pareto dominating policies. *The Journal of Machine Learning Research* 15, 1 (2014), 3483–3512.
- [25] Runzhe Yang, Xingyuan Sun, and Karthik Narasimhan. 2019. A Generalized Algorithm for Multi-Objective Reinforcement Learning and Policy Adaptation. In *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 14610–14621.
- [26] Luisa M Zintgraf, Timon V Kanthers, Diederik M Roijers, Frans Oliehoek, and Philipp Beau. 2015. Quality assessment of MORL algorithms: A utility-based approach. In *Benelearn 2015: proceedings of the 24th annual machine learning conference of Belgium and the Netherlands*.
- [27] Eckart Zitzler and Lothar Thiele. 1999. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE transactions on Evolutionary Computation* 3, 4 (1999), 257–271.
- [28] Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M Fonseca, and Viviane Grunert Da Fonseca. 2003. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on evolutionary computation* 7, 2 (2003), 117–132.